

SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	Assessment:	Industry Problem solving task
Weightage:	30%	Total Marks:	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	G Lokesh Yadav	Reg. No.:	192411061
Section / Batch:	2024-CSE	Date:	19-08-2026

Instructions to Students

- Identify the relevant concepts of cache hierarchy, SRAM, DRAM, MRAM, NAND Flash, RRAM, memory latency, bandwidth, and virtual memory paging.
- Analyze the memory-access requirements of smartphones, cloud servers, autonomous vehicles, edge AI devices, and gaming consoles, considering latency, bandwidth, capacity, and power consumption.
- Apply suitable memory-hierarchy techniques such as cache optimization, appropriate memory technology selection, data locality, paging optimization, and hierarchical memory organization.
- Compute performance measures such as memory access time, latency, bandwidth, throughput, speedup, hit/miss rates, and resource or power utilization where applicable.
- Interpret the results by evaluating performance improvements, memory bottlenecks, technology trade-offs, and the suitability of the proposed memory hierarchy for each application.

QUESTIONS

1. A mobile device manufacturer observes noticeable lag when users switch rapidly between multiple applications. Analyze how L1, L2, and L3 caches and DRAM participate in storing and retrieving frequently accessed application data. Based on latency, capacity, hit rate, and bandwidth requirements, propose a suitable hierarchical memory organization that can reduce application switching delays and improve overall system throughput.
2. A cloud service provider experiences increased latency when processing database queries from a large number of concurrent users. Analyze how cache hierarchy and virtual memory paging influence memory access performance, and compare their roles in reducing processor-memory latency. Recommend suitable memory technologies and memory-management improvements that can minimize page faults, reduce access delays, and improve database throughput.
3. An autonomous vehicle continuously receives large volumes of sensor data from cameras, LiDAR, radar, and other real-time sensing systems. Evaluate the characteristics of SRAM, DRAM, and MRAM in terms of access latency, bandwidth, capacity, volatility, and power consumption. Based on the real-time processing requirements, design an appropriate memory hierarchy that can provide high data-transfer bandwidth while minimizing access latency and supporting reliable sensor-data processing.
4. An edge-based AI device must load machine-learning models quickly while operating under strict power and storage constraints. Analyze the characteristics of NAND Flash, DRAM, and RRAM with respect to access speed, storage capacity, persistence, power consumption, and suitability for AI workloads. Recommend a hierarchical memory organization that enables fast model loading, efficient intermediate-data processing, reduced energy consumption, and improved overall inference performance.
5. A high-performance gaming console experiences frame drops and noticeable delays when loading large game environments containing high-resolution textures, audio, and other assets. Analyze the interaction between L3 cache and DRAM during the retrieval and processing of frequently

accessed game data, considering their latency, capacity, and bandwidth. Propose suitable memory hierarchy enhancements that can reduce data-access bottlenecks, increase memory throughput, and provide smoother frame rendering during large scene transitions.

Code of Conduct

I G Lokesh Yadav-192411061 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: lokesh yadav

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Q. No	Understanding of Industry Problem (20)	Application of Engineering	Solution Design (25)	Use of Tools (15)	Analysis (10)	Professionalism(5)	Total (out of 100)

		Concepts (25)					
1	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 5	/ 100

Course Coordinator Faculty Incharge

QUESTION 1

A mobile device manufacturer observes noticeable lag when users switch rapidly between multiple applications. Analyze how L1, L2, and L3 caches and DRAM participate in storing and retrieving frequently accessed application data. Based on latency, capacity, hit rate, and bandwidth requirements, propose a suitable hierarchical memory organization that can reduce application switching delays and improve overall system throughput.

Solution

When a mobile device switches rapidly between applications, the processor repeatedly accesses instructions, program data, stack information, and operating-system data. A hierarchical memory system is required because a single memory technology cannot simultaneously provide very low latency, large capacity, and high bandwidth.

Role of L1, L2, L3 Cache and DRAM

L1 Cache: It is the smallest and fastest cache and is located closest to the CPU core. It stores the most frequently accessed instructions and data. Its very low access latency makes it important for immediate execution after an application switch.

L2 Cache: It is larger than L1 but slightly slower. It stores data and instructions that are not present in L1 and reduces the number of accesses to slower lower-level memory.

L3 Cache: It is generally larger and shared among multiple CPU cores. It acts as an intermediate buffer between L2 caches and DRAM. It is especially useful when multiple applications or threads share data.

DRAM: DRAM provides the large main-memory capacity required to keep active applications, operating-system components, buffers, and background processes. It has much higher latency than cache memory, but its capacity and bandwidth are much greater.

Performance Comparison

Level	Latency	Capacity	Hit-rate role	Bandwidth
L1	Very low	Very small	Highest for immediate	Very high

			working data	
L2	Low	Small/medium	High after L1 miss	High
L3	Moderate	Large	Catches many L1/L2 misses	High
DRAM	High	Very large	Main-memory fallback	Very high

Proposed Hierarchical Organization

CPU Cores → Private L1 I/D Cache → Private L2 Cache → Shared L3 Cache → High-Bandwidth LPDDR/DRAM

The proposed design uses separate instruction and data paths at L1, a private L2 cache for each core, a larger shared L3 cache, and high-bandwidth low-power DRAM as the main memory. Frequently used application-switching data should remain in the upper cache levels through effective replacement and prefetching policies.

Recommended Improvements

Use sufficient L1 capacity with very low latency for the active working set.

Use private L2 caches to reduce contention between CPU cores.

Use a sufficiently large shared L3 cache to retain data belonging to recently used applications.

Use high-bandwidth LPDDR DRAM so that cache misses and application loading can be serviced quickly.

Use hardware/software prefetching for predictable application data.

Use cache-aware scheduling so that frequently switched applications are preferentially scheduled on cores where their useful cache data remains available.

Avoid unnecessary memory transfers and use efficient cache replacement policies.

Conclusion: A balanced L1–L2–L3–DRAM hierarchy reduces average memory access time by satisfying most requests from fast cache levels. This decreases application switching latency and improves CPU utilization and overall throughput.

QUESTION 2

A cloud service provider experiences increased latency when processing database queries from a large number of concurrent users. Analyze how cache hierarchy and virtual memory paging influence memory access performance, and compare their roles in reducing processor-memory latency. Recommend suitable memory technologies and memory-management improvements that can minimize page faults, reduce access delay, and improve database throughput.

Solution

Cloud database workloads contain frequently reused indexes, query plans, records, buffers, and metadata. With many concurrent users, memory contention increases and cache misses or virtual-memory page faults can significantly increase query latency.

Effect of Cache Hierarchy

L1 cache provides the fastest access to data actively used by the processor.

L2 and L3 caches hold progressively larger working sets and reduce accesses to DRAM.

Database buffer pools can keep frequently accessed database pages in DRAM, avoiding repeated storage accesses.

Good locality of reference improves cache hit rate and reduces average memory access time.

Effect of Virtual Memory Paging

Virtual memory allows each database process to use a large logical address space. The operating system maps virtual pages to physical frames. If a required page is not present in physical memory, a page fault occurs and the operating system must fetch the page from secondary storage. This is extremely slower than cache or DRAM access.

Cache miss: The required data is absent from a cache level and is fetched from the next memory level.

Page fault: The required virtual-memory page is absent from DRAM and must be obtained from storage or another backing location.

Therefore, page faults have a much larger latency penalty than ordinary cache misses.

Recommended Memory and Management Improvements

Use high-capacity DDR5-class DRAM or equivalent high-bandwidth server memory for database working sets.

Use large last-level caches to reduce processor-to-DRAM traffic.

Allocate sufficient RAM so frequently used database pages remain resident.

Use huge pages where appropriate to reduce page-table overhead and Translation Lookaside Buffer (TLB) misses.

Use NUMA-aware memory allocation and scheduling on multi-socket systems.

Tune database buffer pools so hot pages remain in memory.

Use efficient replacement policies and avoid unnecessary swapping.

Use SSD/NVMe storage as fast backing storage when paging or database I/O is unavoidable.

Monitor page-fault rate, cache-miss rate, memory bandwidth, and database buffer-cache hit ratio.

Goal: High cache hit rate + large resident database working set + low page-fault rate → Lower query latency → Higher database throughput

Conclusion: Cache hierarchy mainly reduces CPU-to-memory latency, whereas virtual memory provides capacity and isolation but can introduce large delays when page faults occur. The best design combines large, fast caches, sufficient DRAM, optimized database buffering, and careful virtual-memory management.

QUESTION 3

An autonomous vehicle continuously receives large volumes of sensor data from cameras, LiDAR, radar, and other real-time sensing systems. Evaluate the characteristics of SRAM, DRAM, and MRAM in terms of access latency, bandwidth, capacity, volatility, and power consumption. Based on the real-time processing requirements, design an appropriate memory hierarchy that can provide high data-transfer bandwidth while minimizing access latency and supporting reliable sensor-data processing.

Solution

Autonomous vehicles require deterministic and rapid processing of sensor streams. Camera frames, LiDAR point clouds, radar measurements, object-detection features, and control information must be accessed quickly. Therefore, the memory hierarchy must combine very fast memory for immediate processing with large-capacity

memory for buffering sensor data.

SRAM, DRAM and MRAM Comparison

Technology	Latency	Bandwidth	Capacity	Volatility	Power
SRAM	Very low	Very high	Small	Volatile	High per bit; no refresh
DRAM	Low–moderate	High	Very large	Volatile	Requires refresh
MRAM	Low	High	Medium	Non-volatile	Low standby; no refresh

Proposed Memory Hierarchy

CPU/GPU/NPU Registers → SRAM/L1-L2 Cache → Shared SRAM/L3 Cache → High-Bandwidth DRAM → MRAM/NVM for Persistent/Retained Data

Functional Allocation

Registers and SRAM caches: Store immediate sensor-processing variables, neural-network intermediate values, control loops, and frequently reused data.

DRAM: Store camera frames, LiDAR point clouds, radar buffers, neural-network models, and larger working datasets requiring high capacity and bandwidth.

MRAM: Store non-volatile configuration data, calibration parameters, event logs, selected safety-critical state, and data that should persist without continuous power.

Design Improvements

Use high-bandwidth DRAM close to the processing unit for continuous sensor streams.

Use DMA transfers so sensor interfaces can move data to memory without excessive CPU involvement.

Use multiple memory channels to increase aggregate bandwidth.

Use SRAM caches and on-chip buffers for low-latency real-time processing.

Use double buffering or ring buffers so one sensor buffer can be processed while another receives new data.

Use MRAM for non-volatile information that must survive power loss and should not require refresh.

Apply data compression or selective buffering when raw sensor data exceeds available memory bandwidth.

Conclusion: SRAM should serve latency-critical on-chip operations, DRAM should provide high-capacity high-bandwidth storage for active sensor streams, and MRAM should provide reliable non-volatile storage for retained information. This hierarchy balances speed, capacity, power, and reliability.

QUESTION 4

An edge-based AI device must load machine-learning models quickly while operating under strict power and storage constraints. Analyze the characteristics of NAND Flash, DRAM, and RRAM with respect to access speed, storage capacity, persistence, power consumption, and suitability for AI workloads. Recommend a hierarchical memory organization that enables fast model loading, efficient intermediate-data processing, reduced energy consumption, and improved overall inference performance.

Solution

Edge AI devices such as smart cameras, robots, and embedded assistants have limited power, memory, and storage.

AI inference requires rapid access to model weights, activations, intermediate tensors, and input data. A hierarchy is therefore required to place data according to frequency and speed of access.

NAND Flash, DRAM and RRAM Comparison

Technology	Access speed	Capacity	Persistence	Power	AI role
NAND Flash	Slow compared with RAM	High	Non-volatile	Low standby	Model/storage repository
DRAM	Fast	High	Volatile	Moderate	Model weights, activations, working data
RRAM	Fast; suitable for dense non-volatile storage	Medium-high	Non-volatile	Low standby	Model storage / compute-near-memory potential

Recommended Hierarchy

AI Accelerator/CPU → On-chip SRAM/Cache → DRAM → RRAM → NAND Flash

Operation

Store the complete AI model and less frequently used files in NAND Flash.

At boot or model-selection time, copy the required model into faster DRAM or RRAM.

Keep frequently reused model weights and active tensors in on-chip SRAM/cache.

Perform matrix operations and intermediate-data processing using the accelerator while data remains close to the compute units.

Use DMA and tiled data movement to reduce CPU overhead and unnecessary memory transfers.

Energy and Performance Improvements

Quantize model weights from higher precision to INT8 or other suitable low-precision formats where accuracy permits.

Use model pruning or compression to reduce storage and memory traffic.

Use RRAM/NVM for retained models to reduce repeated loading from slower storage.

Use on-chip SRAM for the hottest data to reduce DRAM accesses.

Use batch/tile processing that matches the available on-chip memory.

Reduce unnecessary copies between NAND, DRAM, and accelerator memory.

Conclusion: NAND Flash should provide large persistent storage, DRAM should act as the main high-speed working memory, and RRAM can provide fast non-volatile retention and potentially support compute-near-memory techniques. A small on-chip SRAM/cache layer is essential for low-latency inference and reduced energy consumption.

QUESTION 5

A high-performance gaming console experiences frame drops and noticeable delays when loading large game environments containing high-resolution textures, audio, and other assets. Analyze the interaction between L3 cache and DRAM during the retrieval and processing of frequently accessed game data, considering their latency, capacity, and bandwidth. Propose suitable memory hierarchy enhancements that can reduce data-access bottlenecks, increase memory throughput, and provide smoother frame rendering during large scene transitions.

Solution

Modern games continuously stream textures, geometry, audio, animation data, shaders, and world-state information. During large scene transitions, the amount of data requested by the CPU/GPU increases sharply. If the required data is not available in cache or sufficiently fast memory, stalls can occur and cause frame drops.

Interaction Between L3 Cache and DRAM

- The L3 cache acts as a large last-level cache between the processor and DRAM.
- Frequently reused game data can remain in L3, allowing faster access than going to DRAM.
- When data is absent from L3, a cache miss causes a DRAM access.
- DRAM provides much greater capacity than L3 and stores the larger active game working set.
- High DRAM bandwidth is essential when large textures and other assets are streamed into the processor or GPU memory.
- Repeated misses can increase memory traffic and cause CPU/GPU stalls, which can lead to frame-time spikes.

Latency, Capacity and Bandwidth

Feature	L3 Cache	DRAM	Effect on gaming
Latency	Lower	Higher	L3 reduces waiting time for hot data
Capacity	Smaller	Much larger	DRAM stores large game assets
Bandwidth	Very high	High to very high	DRAM must sustain large streaming transfers

Proposed Memory Hierarchy Enhancements

CPU/GPU Registers → L1/L2 Cache → Large Shared L3 Cache → High-Bandwidth DRAM → Fast SSD/NVMe Storage

- Increase the effective L3 cache capacity so more frequently reused game data remains close to the processor.
- Use high-bandwidth multi-channel DRAM to sustain texture and asset streaming.
- Use intelligent prefetching to load predictable assets before they are required.
- Use asynchronous asset streaming so data loading overlaps with rendering instead of blocking the frame pipeline.
- Use DMA/direct memory transfers to reduce CPU involvement in large data movement.
- Use fast NVMe SSD storage as the persistent asset repository and stream data into DRAM before it is needed.
- Compress textures and other assets where suitable to reduce memory bandwidth requirements.
- Use efficient cache replacement policies so important rendering data is retained.

Frame Rendering Strategy

**SSD/NVMe → DRAM Prefetch Buffer → L3/Cache → CPU/GPU → Frame Buffer
→ Display**

During a scene transition, the system should predict upcoming assets and prefetch them from storage into DRAM. Frequently reused processing data can then be retained in cache. This hides storage latency, reduces DRAM stalls, and helps maintain a stable frame time.

Conclusion: A larger and efficient L3 cache combined with high-bandwidth DRAM, predictive prefetching, asynchronous streaming, and fast NVMe storage can reduce memory bottlenecks. The result is faster scene loading, fewer CPU/GPU stalls, and smoother frame rendering.

Overall Conclusion

The five scenarios demonstrate that computer systems achieve high performance by using a hierarchy rather than relying on a single memory technology. Small, fast memories such as SRAM and cache reduce latency, while larger DRAM provides the main working capacity. Non-volatile memories such as NAND Flash, MRAM, and RRAM provide persistent storage or retention. Effective cache policies, prefetching, DMA, sufficient DRAM capacity, high memory bandwidth, and optimized virtual-memory management are essential for reducing access delays and improving system throughput.